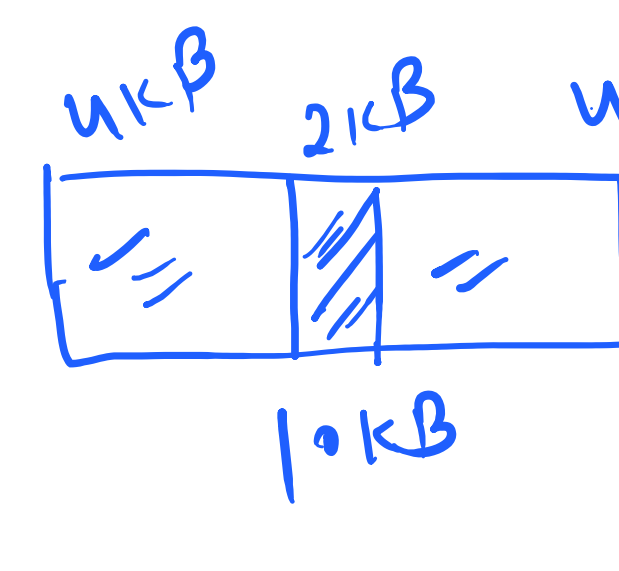
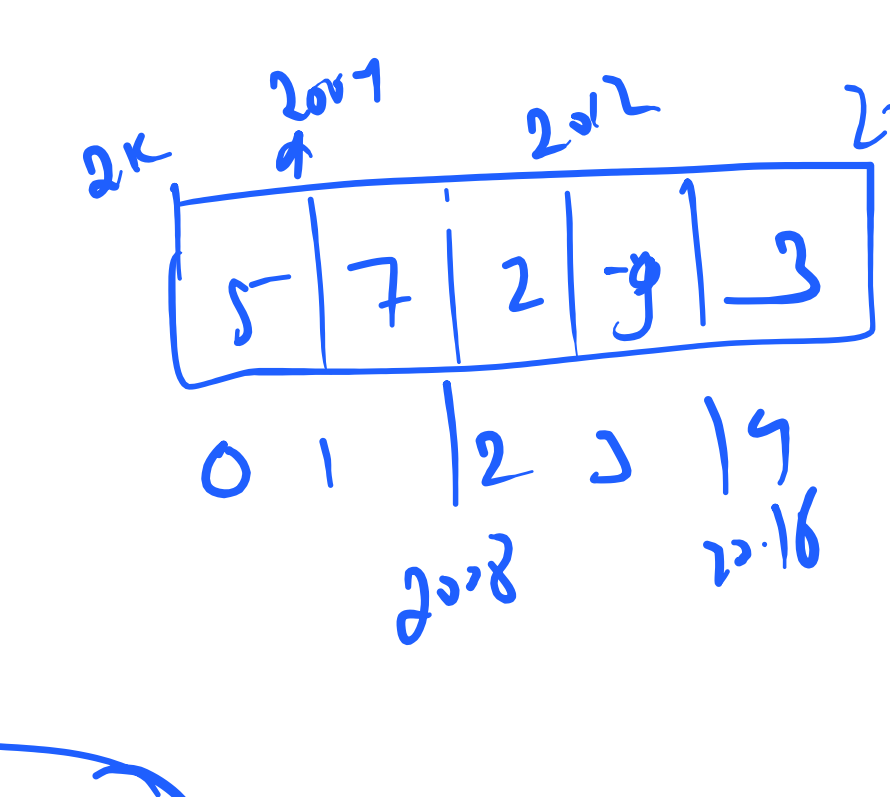


LinkedList

non-continuous



P = 6KB

0-8

memory increment

Control

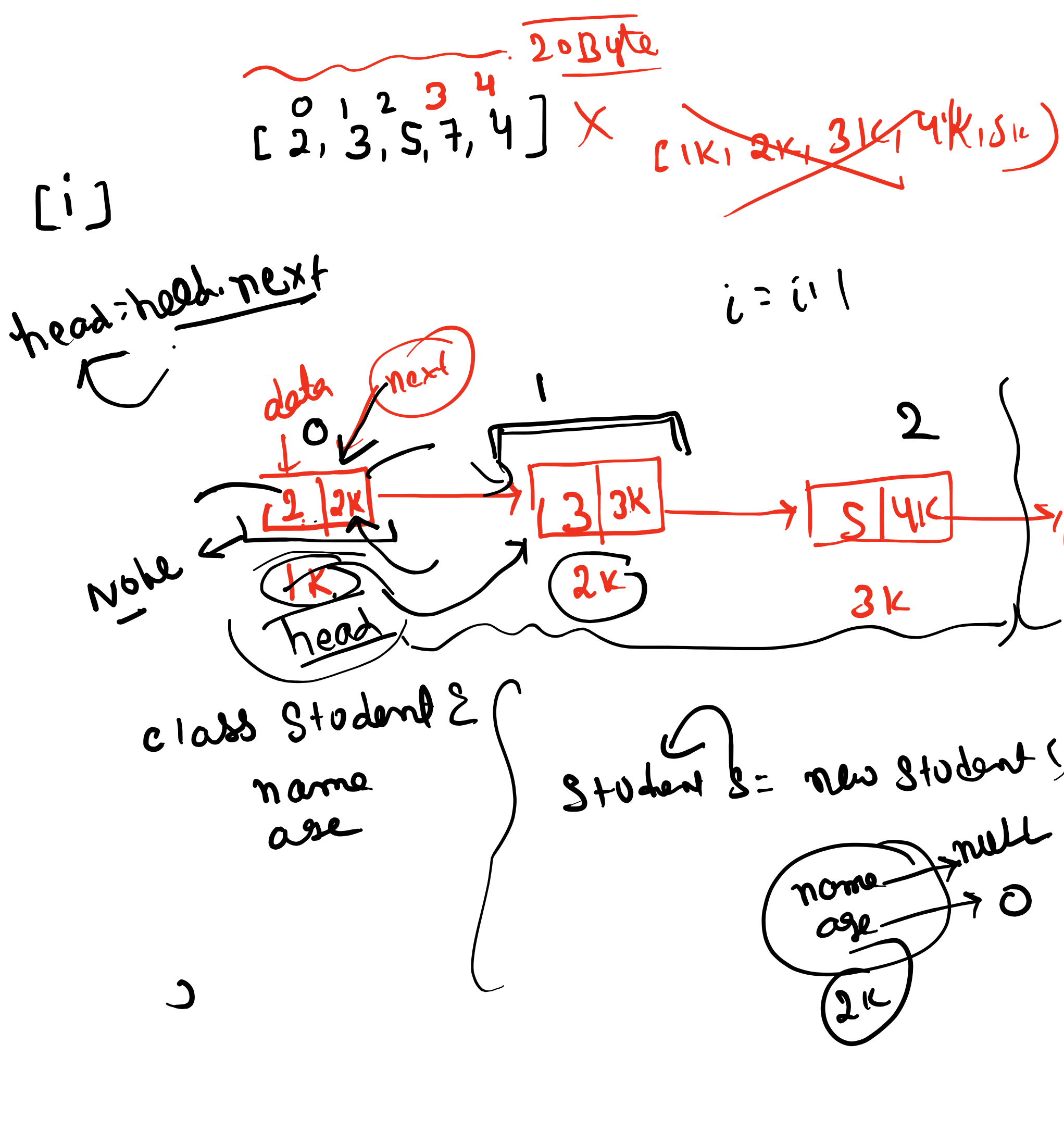
min-ops

Passing sem

FT BT WT

tail -> null

tail.next -> null

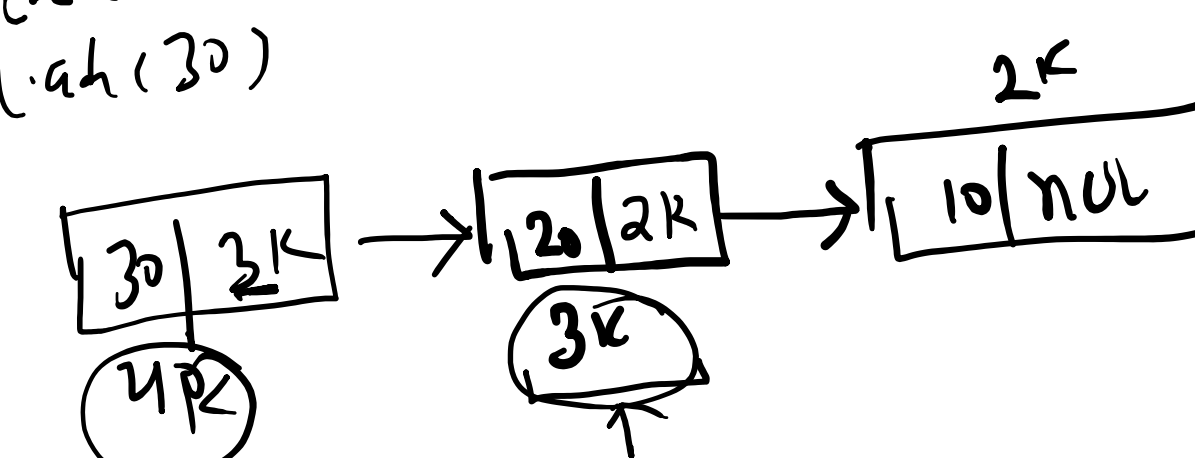


class Node {
int val;
Node next;
}

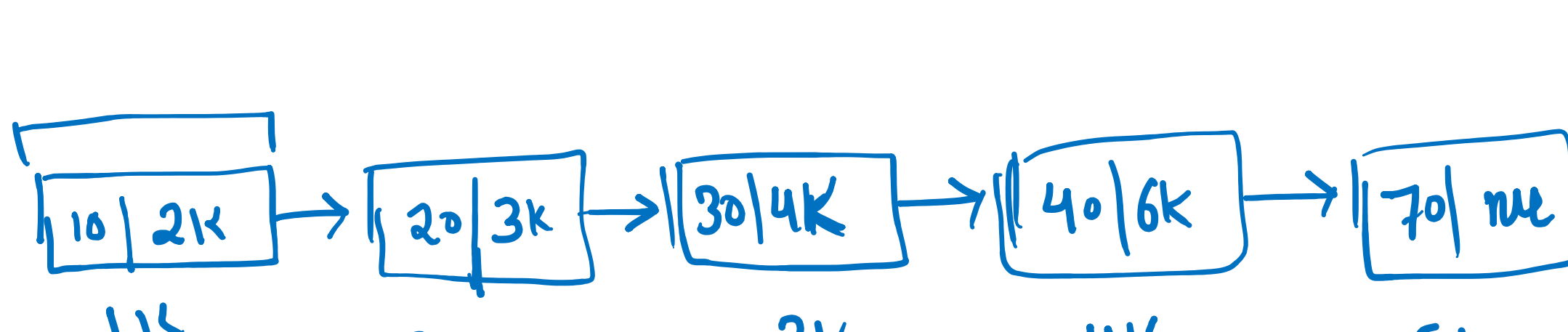
- 1) add first
- 2) add last
- 3) index at
- 4) get first
- 5) get last
- 6) get at index
- 7) remove first
- 8) remove last
- 9) remove at index
- 10) get display
- 11) size

```
public void addfirst(int item) {  
    Node nn = new Node(item);  
    if (size == 0) {  
        head = nn;  
        tail = nn;  
        size++;  
    }  
    else {  
        nn.next = head;  
        head = nn;  
        size++;  
    }  
}
```

add(10)
add(20)
add(30)



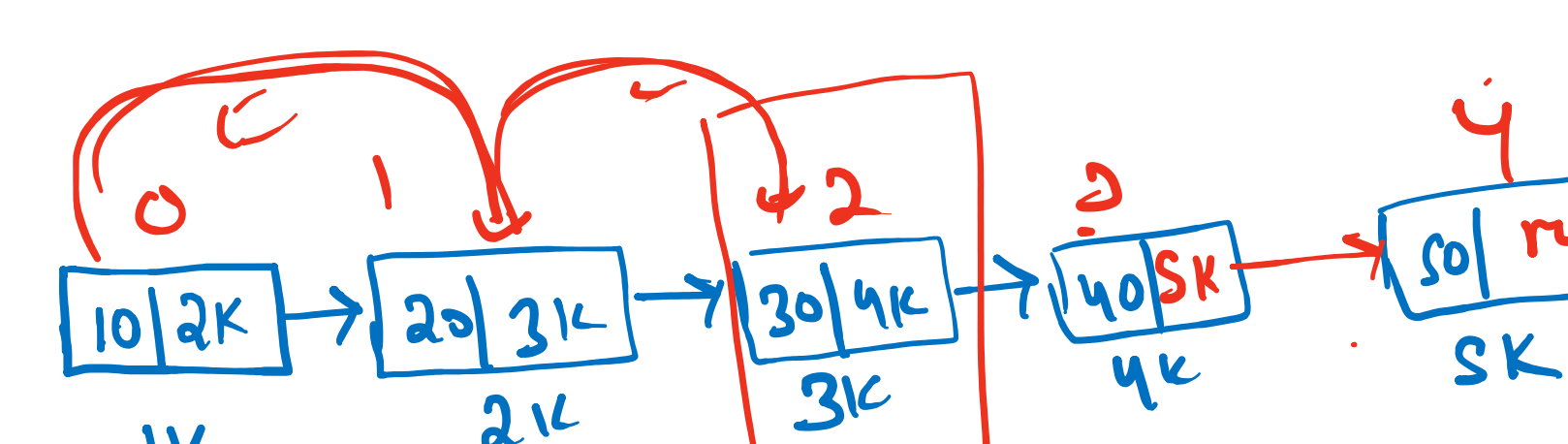
nn.next = head
head = nn
size++



Node temp = head
while (temp != null) {
 // do something
 temp = temp.next
}

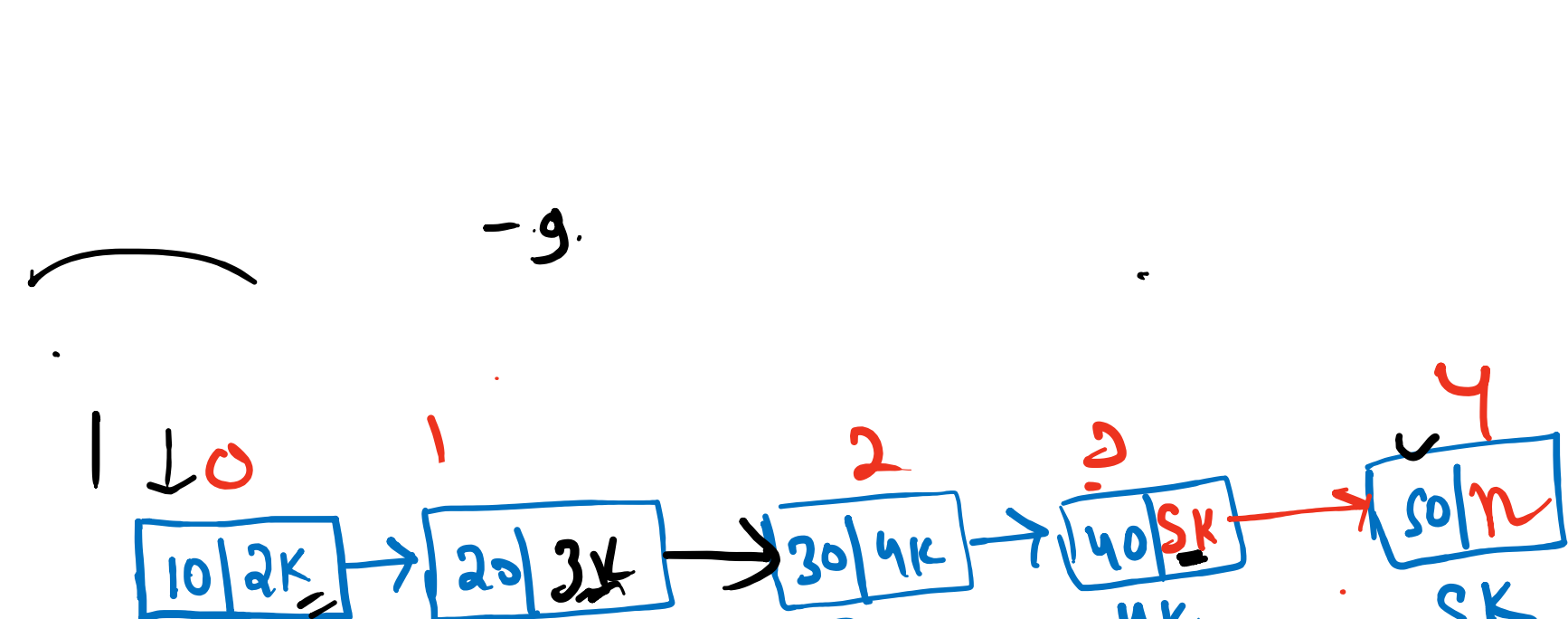
10 20 30 40 70

```
public void addlast(int item) {  
    if (size == 0) {  
        addfirst(item);  
    }  
    else {  
        Node nn = new Node(item);  
        temp = head;  
        while (temp.next != null) {  
            temp = temp.next;  
        }  
        temp.next = nn;  
        size++;  
    }  
}
```

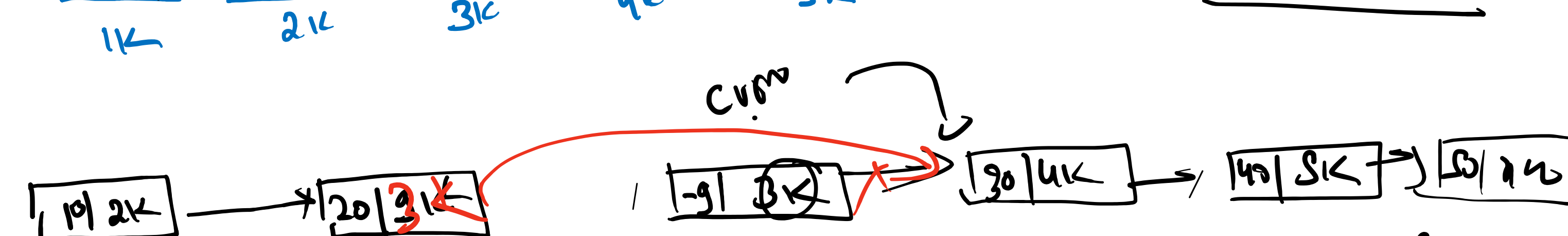


temp = head
while (temp.next != null) {
 temp = temp.next;
}

tail.next = nn
tail = nn
size++



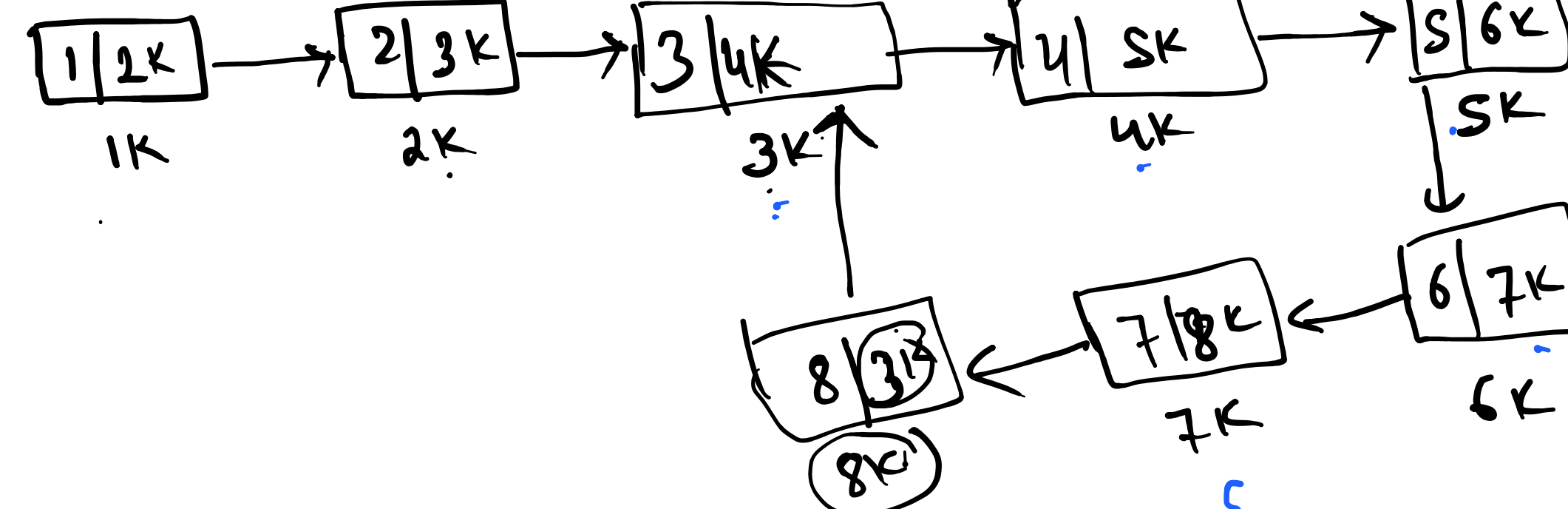
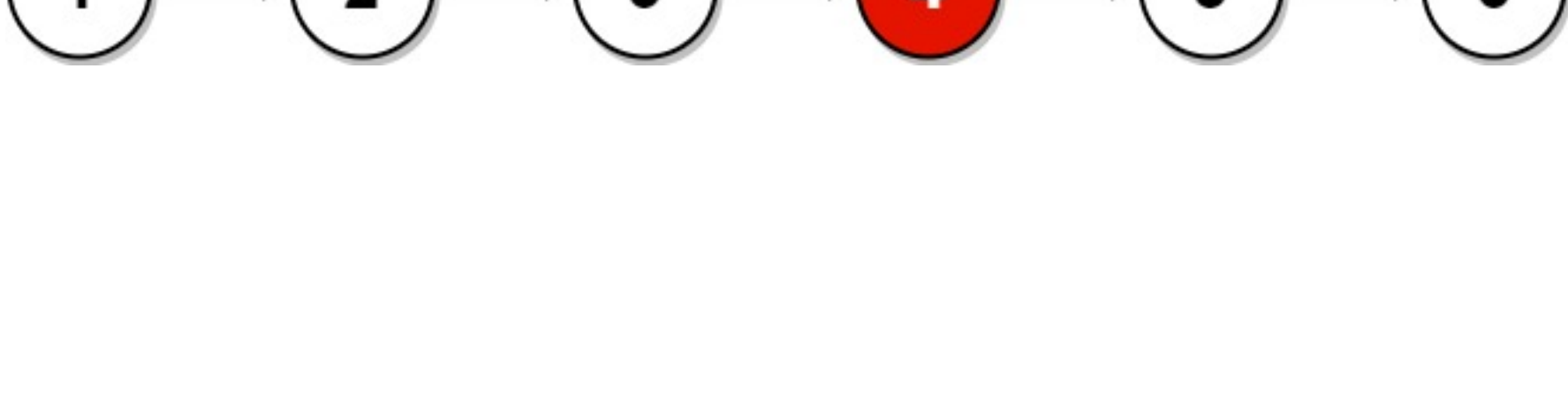
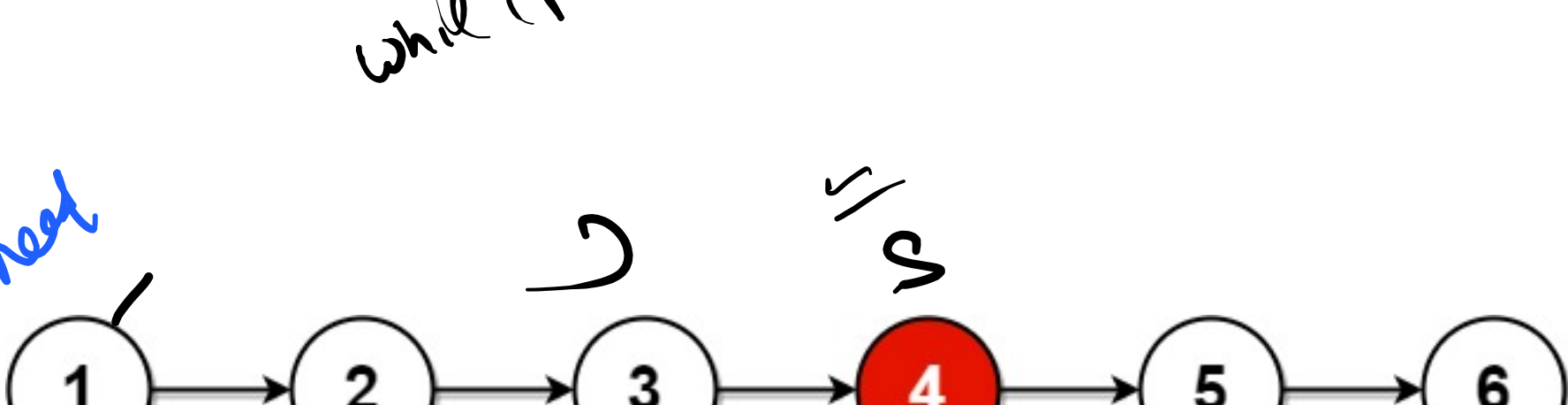
nn.next = prev.next
prev.next = nn



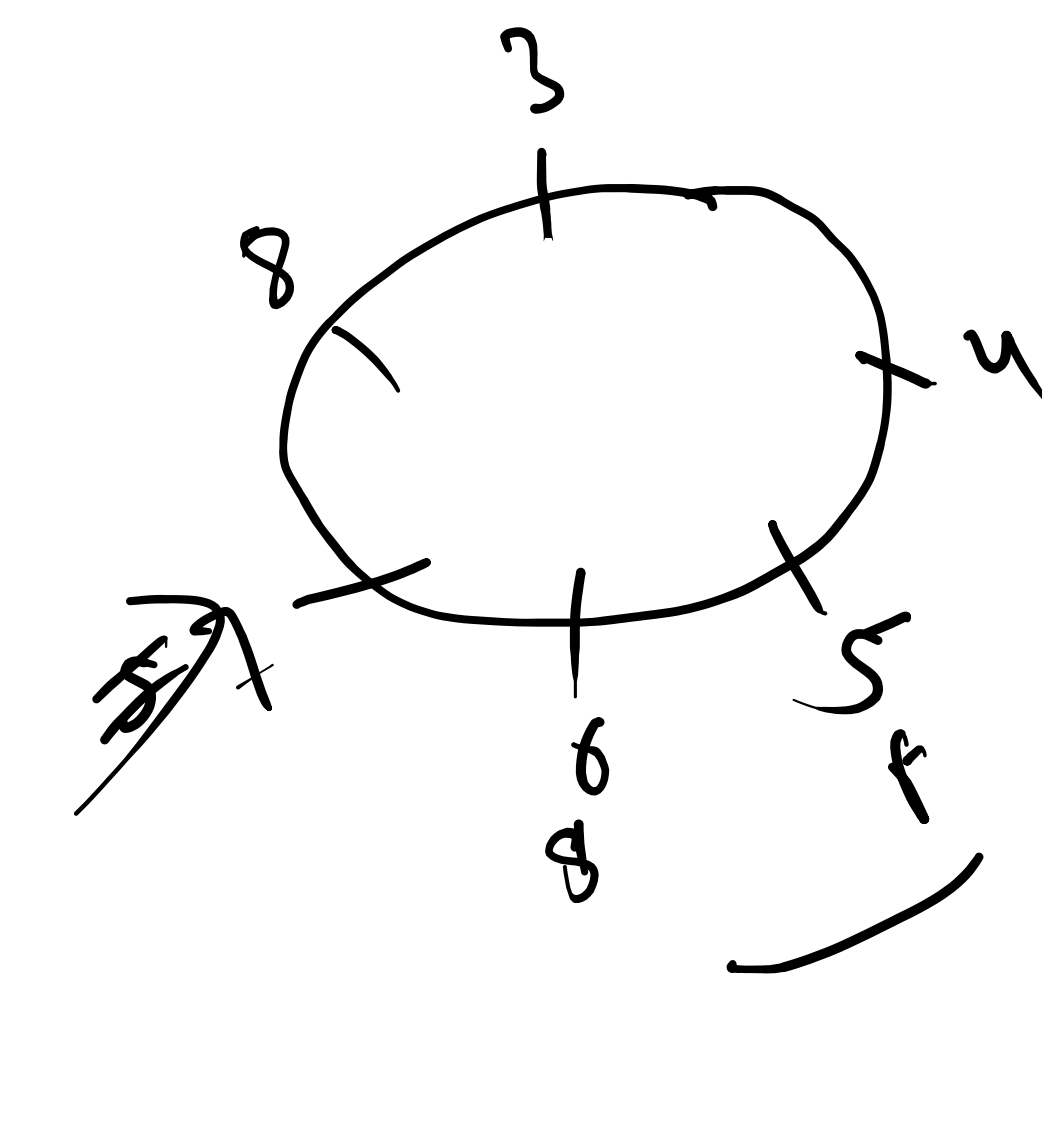
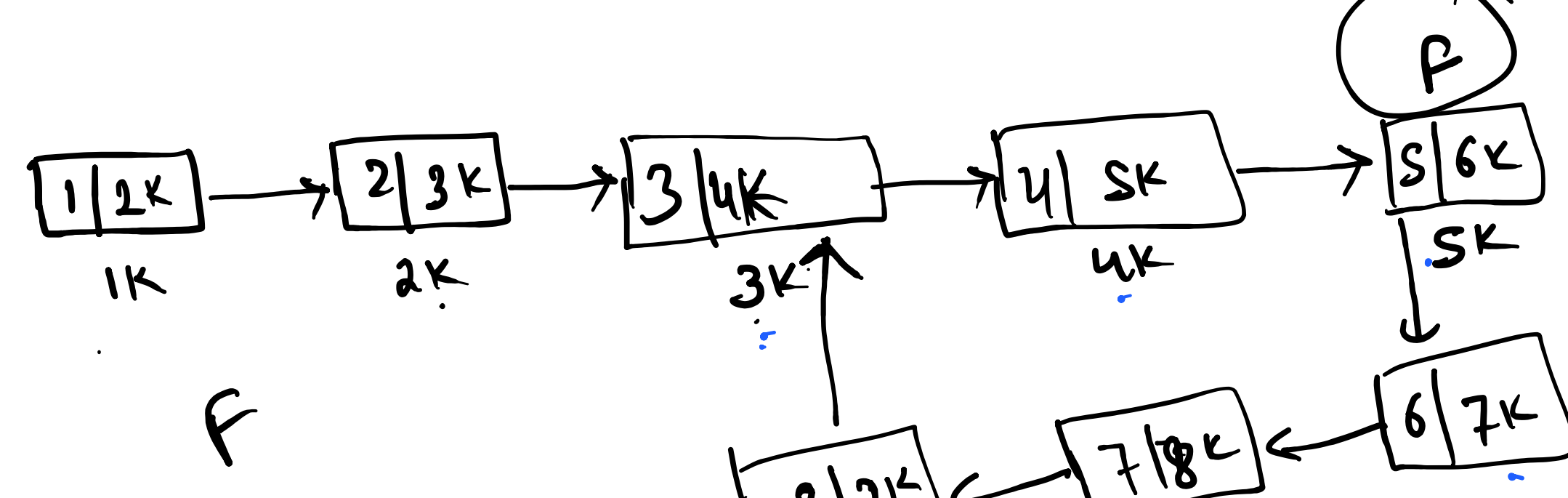
prev = getNth(10)
curr = prev.next
prev.next = curr.next
curr.next = prev

Node curr = head
head = head.next
curr.next = head

while (fast != null && fast.next != null) {
 slow = slow.next;
 fast = fast.next.next;
}



```
ListNode slow = head;  
ListNode fast = head;  
while (fast != null && fast.next != null) {  
    slow = slow.next;  
    fast = fast.next.next;  
}  
return slow;
```



l1 -> 8
l2 -> 8
|l1 - l2| = 0
Size x

